

Universidad del Valle de Guatemala

Facultad de Ingeniería · Departamento de Ciencias de la Computación

CC3084 – Data Science · Sección 10 · Semestre II 2026

Proyecto #2 · Resultados

MAP — Charting Student Math Misunderstandings

Clasificación de concepciones erróneas matemáticas en explicaciones estudiantiles mediante Procesamiento del Lenguaje Natural

Grupo #1

Jorge Gabriel Palacios Sales

Pablo Daniel Barillas Moreno

Roberto Emiliano Otoniel Camposeco Torres

Abby Sofia Donis Agreda

Guatemala, 13 de octubre de 2026

Índice

Índice 2
1. Introducción 4
2. Objetivos 5
2.1 Objetivo general 5
2.2 Objetivos específicos 5
3. Marco teórico 6
3.1 Procesamiento del Lenguaje Natural 6
3.2 Actividades de preprocesamiento 6
3.3 Representación TF-IDF y n-gramas 7
3.4 Algoritmos de clasificación para PLN 7
3.5 Métricas de evaluación 8
3.6 Validación y fuga de información 8
4. Metodología 9
4.1 Pasos seguidos 9
4.2 Selección de los conjuntos de entrenamiento y prueba 9
4.3 Selección de los algoritmos 10
4.4 Representación de entrada 10
4.5 Herramientas utilizadas 11
5. Resultados y análisis de resultados 12
5.1 Características del conjunto de datos original 12
5.2 Limpieza y preprocesamiento 12
5.3 Análisis exploratorio 13
5.4 Ajuste de hiperparámetros 15
5.5 Comparación de los algoritmos 17
5.6 Ablación del contexto de entrada 19
5.7 Generalización a preguntas no vistas (validación B) 19
5.8 Calibración y abstención 20
5.9 Análisis de errores y selección del modelo 21
5.10 Aplicación 23
6. Conclusiones 28
Trabajo futuro 29

Referencias 30
Enlace al Repositorio de GitHub 31
Enlace al documento del informe en OneDrive 31

1. Introducción

Cuando un estudiante resuelve mal un problema de matemáticas, la respuesta incorrecta es solo el síntoma. Lo que realmente orienta al docente es el porqué: el razonamiento defectuoso, la concepción errónea o "misconception", que produjo esa respuesta. Del mismo modo, una respuesta correcta no garantiza un razonamiento correcto: un estudiante puede acertar por casualidad o mediante una regla equivocada que funciona en ese caso particular (Rushton, 2018).

El reto de Kaggle "MAP - Charting Student Math Misunderstandings" (Kaggle, 2025; Rittle-Johnson et al., 2025) propone automatizar parte de ese diagnóstico. Cada observación contiene una pregunta de opción múltiple, la opción elegida por el estudiante y una breve explicación escrita de su razonamiento. A partir de ello se debe predecir una etiqueta compuesta `Category:Misconception` que indica (i) si la opción es correcta, (ii) si el razonamiento es correcto, contiene una concepción errónea o no es evaluable, y (iii) cuál es la concepción errónea específica, cuando la hay.

En la fase anterior del proyecto se realizó un análisis exploratorio completo y una revisión bibliográfica de algoritmos. Este informe reúne ambos trabajos y presenta la fase de resultados: la selección de algoritmos, la construcción y ajuste de varios modelos, su evaluación con métricas adecuadas para clasificación multiclase desbalanceada y la discusión comparativa que permite escoger el modelo final.

Problema a resolver

Revisar manualmente miles de explicaciones abiertas exige tiempo y conocimiento pedagógico especializado. Se busca un clasificador que, dada la pregunta, la opción y la explicación, proponga las tres etiquetas más probables ordenadas por confianza, de manera que el docente pueda priorizar la revisión. El sistema se concibe como apoyo a la decisión y no como diagnóstico automático definitivo del estudiante.

Pregunta científica

¿Qué combinación de representación textual, contexto de la pregunta y algoritmo de clasificación permite ordenar con mayor precisión las categorías y las concepciones erróneas presentes en explicaciones matemáticas breves, y cuánto de ese desempeño se conserva ante preguntas no vistas?

2. Objetivos

2.1 Objetivo general

Construir, ajustar y comparar modelos de aprendizaje automático supervisado que clasifiquen explicaciones matemáticas de estudiantes en la etiqueta compuesta `Category:Misconception` del reto MAP, fundamentando las decisiones de preprocesamiento, validación y selección del modelo en el análisis exploratorio y en la literatura revisada.

2.2 Objetivos específicos

1. Caracterizar la estructura, calidad y desbalance del conjunto de datos y derivar de ello las decisiones de limpieza y preprocesamiento.
2. Revisar la literatura sobre calificación automática de respuestas cortas y detección de concepciones erróneas para seleccionar algoritmos candidatos.
3. Entrenar al menos cuatro modelos (una línea base sin texto y tres clasificadores sobre TF-IDF) y ajustar sus hiperparámetros con validación cruzada agrupada.
4. Evaluar los modelos con MAP@3, exactitud, macro-F1 y F1 ponderado, además de tiempos de entrenamiento y predicción.
5. Diseñar y evaluar un modelo jerárquico que aproveche la estructura del problema descubierta en el análisis exploratorio.
6. Medir la capacidad de generalización a preguntas no vistas mediante validación agrupada por pregunta.

3. Marco teórico

3.1 Procesamiento del Lenguaje Natural

El **Procesamiento del Lenguaje Natural** (PLN o NLP) es el área de la inteligencia artificial y la lingüística computacional que desarrolla métodos para que una computadora represente, analice, interprete y genere lenguaje humano. Incluye tareas como la clasificación de textos, la extracción de información, la traducción automática y la respuesta a preguntas. Este proyecto aborda una tarea de **clasificación de texto corto multiclase con ranking**: cada explicación (unas 13 palabras de mediana) se asigna a una de 65 clases, y se devuelven las tres más probables.

En educación, el PLN se ha aplicado a la **calificación automática de respuestas cortas** (*Automatic Short Answer Grading*, ASAG). La evolución del campo pasa por cuatro etapas: reglas y similitud con respuestas modelo, interpretables pero frágiles ante paráfrasis; representaciones léxicas como TF-IDF con clasificadores lineales, competitivas y baratas para texto corto; modelos de lenguaje preentrenados como BERT y DeBERTa, que capturan contexto bidireccional; y adaptación al dominio, que incorpora vocabulario y patrones específicos (Burrows et al., 2015; Morley et al., 2026). La literatura converge en cuatro decisiones: usar el contexto completo (pregunta y respuesta además de la explicación), aprovechar la transferencia cuando hay clases con pocos ejemplos, considerar la adaptación al dominio y tratar la calibración, la robustez y la supervisión humana como requisitos (Zhang et al., 2020; Bonthu et al., 2024; Schneider et al., 2023; Filighera et al., 2024).

3.2 Actividades de preprocesamiento

- **Normalización:** conversión a minúsculas y unificación de espacios.
- **Limpieza de marcado:** las opciones venían en LaTeX (96 %), por ejemplo `\( \frac{1}{3} \)`, y se convirtieron a texto plano (`1/3`).
- **Tokenización:** división del texto en unidades. Para TF-IDF se definió un patrón que conserva números y operadores (`+ - * / = < > %`); los *Transformers* usan su propio tokenizador de subpalabras.
- **Palabras vacías (*stopwords*):** no se eliminaron, porque palabras como *not* o *than* cambian el sentido matemático.
- **Números, operadores, signos y negaciones:** se conservaron. El 78.3 % de las explicaciones contiene dígitos; son la principal señal matemática.
- **Corrección ortográfica y lematización:** no se aplicaron; los n-gramas de caracteres y los tokenizadores de subpalabras toleran errores como "tree nineth".
- **Vectorización:** TF-IDF para los modelos lineales y *embeddings* contextuales para los *Transformers*.

3.3 Representación TF-IDF y n-gramas

TF-IDF (*Term Frequency – Inverse Document Frequency*) asigna a cada término t de un documento d un peso proporcional a su frecuencia en el documento e inversamente proporcional al número de documentos que lo contienen (Salton & Buckley, 1988):

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot [\log((1 + N) / (1 + \text{df}(t))) + 1]$$

donde N es el número de documentos y df(t) el número que contiene t; se usó la variante sublineal 1 + log(tf). Un **n-grama** es una secuencia de n unidades consecutivas: los de palabras (n = 1, 2) capturan expresiones como "divided by", y los de caracteres (n = 2 a 5) capturan fragmentos como "3/9" y dan robustez ortográfica.

3.4 Algoritmos de clasificación para PLN

- **Prior por pregunta y opción.** Ordena las etiquetas por su frecuencia en entrenamiento dentro del mismo par (pregunta, opción). No lee el texto: mide cuánto se logra solo con el contexto.
- **Regresión logística multinomial.** Estima $P(y = k \mid x) = \exp(w_k \cdot x) / \Sigma_j \exp(w_j \cdot x)$ minimizando la entropía cruzada con regularización L2 de intensidad 1/C. Entrega probabilidades, necesarias para ordenar candidatos y aplicar un umbral de abstención.
- **Máquina de vectores de soporte (SVM) lineal.** Busca para cada clase el hiperplano de máximo margen (Cortes & Vapnik, 1995); es muy eficaz en espacios dispersos de alta dimensión. No produce probabilidades: el ranking se construye con la distancia al hiperplano.
- **Modelo jerárquico.** Descompone la predicción según la estructura de la etiqueta: la veracidad (si la opción es correcta) se determina con la pregunta y la opción, y el tipo de razonamiento y la concepción errónea se restringen a las etiquetas compatibles con esa veracidad y observadas en la misma pregunta.
- **Transformers.** Arquitectura basada en mecanismos de atención que representa cada token en función de todos los demás. BERT (Devlin et al., 2019) se preentrena con texto no etiquetado y se ajusta a la tarea supervisada (*fine-tuning*). **DeBERTa** (He et al., 2021) mejora a BERT con atención desacoplada —contenido y posición se codifican por separado— y un decodificador de máscara mejorado; su versión v3 es una de las más competitivas en clasificación. La entrada contextual concatena pregunta, opción y explicación, siguiendo la evidencia de que el contexto de la pregunta mejora la representación (Zhang et al., 2020; Zhu et al., 2022).
- **Adaptación al dominio. MathBERT** (Shen et al., 2021) es un BERT preentrenado con textos de educación matemática; el preentrenamiento específico puede capturar vocabulario y estructuras del dominio (Bonthu et al., 2024).
- **Ensamble calibrado.** Combina las probabilidades de modelos complementarios, $p_E = \alpha \cdot p_{\text{Transformer}} + (1 - \alpha) \cdot p_{\text{lineal}}$, con α elegido en validación (Sahu & Bhowmick, 2020; del Gobbo et al., 2023).

3.5 Métricas de evaluación

- **MAP@3:** métrica oficial. De tres etiquetas ordenadas, un acierto en la primera posición vale 1, en la segunda 1/2 y en la tercera 1/3.
- **Exactitud y exactitud top-3:** proporción de casos con la etiqueta real en la primera posición o entre las tres primeras. La exactitud es engañosa con clases desbalanceadas.
- **Macro-F1 y F1 ponderado:** el F1 combina precisión y exhaustividad; el macro-F1 promedia todas las clases con igual peso, por lo que es sensible a las clases raras.
- **Calibración y ECE:** un modelo está calibrado si, entre los casos en que dice tener 80 % de confianza, acierta el 80 %. El *Expected Calibration Error* (ECE) promedia la diferencia entre confianza y exactitud por intervalos. El **escalado de temperatura** divide los logits de una red entre un escalar T ajustado en validación y es un método simple y eficaz para calibrar redes neuronales (Guo et al., 2017).
- **Abstención:** el sistema solo acepta la predicción si su confianza supera un umbral τ; en caso contrario deriva el caso al docente. La curva cobertura–exactitud muestra el compromiso.
-

3.6 Validación y fuga de información

La **validación cruzada** rota el conjunto de validación entre k pliegues; la variante **estratificada** conserva la proporción de clases y la **agrupada** (*GroupKFold*) mantiene cada grupo en un solo pliegue. Hay **fuga de información** cuando el modelo ve en entrenamiento información del conjunto de evaluación —por ejemplo, textos duplicados o la misma pregunta—, lo que infla las métricas. Padó et al. (2024) muestran que los *Transformers* pueden ser robustos en preguntas vistas y no superar a otros modelos en preguntas no vistas, por lo que ambos escenarios deben medirse por separado.

4. Metodología

4.1 Pasos seguidos

1. Revisión bibliográfica de algoritmos de ASAG y detección de concepciones erróneas.
2. Análisis exploratorio (`notebooks/01_analisis_exploratorio.ipynb`).
3. Preprocesamiento derivado de los hallazgos y particiones sin fuga (`map_modelos.py`).
4. Líneas base TF-IDF con búsqueda de hiperparámetros, ablación del contexto de entrada y validación por preguntas no vistas (`notebooks/02_modelado.py`).
5. Ajuste fino de DeBERTa-v3 y MathBERT en GPU (`notebooks/03_transformer.py`, ejecutado en Kaggle mediante `kaggle/construir_kernels.py`).
6. Calibración, ensamble, abstención y figuras (`notebooks/04_ensamble.py`).
7. Comparación y selección del modelo; construcción de la aplicación (`app/app.py`).

La complejidad se adoptó de forma incremental —línea base, *Transformer*, adaptación al dominio y ensamble—, con el criterio de **incorporar un modelo más costoso solo si mejora el MAP@3 y el macro-F1 bajo el mismo protocolo de validación**.

4.2 Selección de los conjuntos de entrenamiento y prueba

El `test.csv` publicado tiene solo 3 filas (el conjunto real solo era accesible dentro de la competencia, ya cerrada), por lo que toda la evaluación se construyó particionando `train.csv`. Se definieron dos protocolos complementarios:

- **Validación A — preguntas conocidas.** `StratifiedGroupKFold` (5 pliegues, semilla 42): un pliegue es la **prueba** (7 339 filas, 20 %) y el resto el **entrenamiento** (29 357 filas). Es estratificada por la etiqueta compuesta y agrupa las explicaciones idénticas de una misma pregunta (970 repetidas), para que ninguna copia quede a ambos lados. Dentro del entrenamiento se reservó un 10 % de **validación** (2 936 filas) para detener el entrenamiento de los *Transformers*, calibrar las probabilidades y fijar el peso del ensamble. Responde a: ¿funciona con respuestas nuevas a preguntas conocidas?
- **Validación B — preguntas no vistas.** `GroupKFold` por `QuestionId` (3 pliegues): cada pliegue deja fuera 5 preguntas completas. Responde a: ¿transfiere a preguntas nuevas sin memorizar el identificador? Como 32 de las 35 concepciones erróneas existen en una sola pregunta, esta prueba es deliberadamente severa.

Como el reparto aleatorio de `StratifiedGroupKFold` cambia entre versiones de scikit-learn, los índices de las particiones se guardaron en `results/particiones.npz` y se reutilizaron en Kaggle, de modo que **todos los modelos se evalúan exactamente sobre las mismas filas**.

El conjunto de prueba se usó una sola vez por modelo. Los hiperparámetros de los modelos lineales se eligieron con validación cruzada interna de 3 pliegues; los de los *Transformers*, con la partición de validación. **Ablación del contexto de entrada.** Para medir cuánto aporta cada fuente de información se comparó la regresión logística con tres entradas: texto solo, texto + pregunta y texto + pregunta + opción.

4.3 Selección de los algoritmos

Tabla 1. Algoritmos seleccionados por nivel y justificación.

Nivel	Modelo	Razón de la selección
Referencia	Prior (pregunta + opción)	El par (pregunta, opción) determina la veracidad en 58 de 60 casos; mide cuánto se logra sin leer el texto.
Línea base	TF-IDF + SVM lineal	Referencia clásica de ASAG, barata y competitiva en texto corto (Burrows et al., 2015).
Línea base	TF-IDF + regresión logística jerárquica	Interpretable y con probabilidades; explota la estructura jerárquica de la etiqueta.
Principal	DeBERTa-v3-base	Contexto bidireccional con atención desacoplada; entrada pregunta + opción + explicación (He et al., 2021; Zhu et al., 2022).
Dominio	MathBERT	Vocabulario y patrones de educación matemática; se adopta solo si mejora a DeBERTa, en especial en clases raras y preguntas nuevas (Shen et al., 2021).
Salida	Ensamble calibrado con ranking top-3	Combina evidencia léxica y contextual; optimiza MAP@3 sin ocultar la incertidumbre (Sahu & Bhowmick, 2020; Guo et al., 2017).

Uso de `QuestionId`. El análisis exploratorio advirtió que el identificador de la pregunta puede convertirse en un atajo: el modelo memoriza qué concepciones corresponden a cada pregunta en lugar de leer el razonamiento. Aun así se incluyó en la línea base, porque la aplicación se usa sobre el banco conocido de 15 preguntas; su costo se cuantificó explícitamente con la ablación y con la validación B (secciones 5.6 y 5.7). Los *Transformers* no reciben el identificador, sino el **texto** de la pregunta, que sí es transferible.

Se descartaron como solución principal los modelos generativos de gran tamaño (LLM) consultados por API: introducen variabilidad de versión, costo, latencia y riesgos de privacidad, y dificultan repetir exactamente los experimentos (Grévisse, 2024).

4.4 Representación de entrada

- **Modelos lineales:** TF-IDF de palabras (1-2 gramas, frecuencia mínima 2) y de caracteres (2-5 gramas, máximo 120 000 rasgos) sobre la explicación normalizada, más codificación *one-hot* de `QuestionId` y del par (pregunta, opción).
- **Transformers:** la cadena `Question: <enunciado> Answer: <opción> Explanation: <explicación>`, truncada a 160 tokens (el percentil 95 es de 80).

4.5 Herramientas utilizadas

- **Recursos de cómputo.** Los modelos lineales, la calibración, el ensamble y la aplicación se ejecutaron en una computadora portátil **Apple M3 (8 núcleos) con 8 GB de RAM**. Un primer intento de ajustar DeBERTa-v3-small en la GPU integrada (MPS) agotó la memoria unificada: una época tomaba horas por el intercambio con disco. Por ello los *Transformers* se entrenaron en **Kaggle con GPU NVIDIA** (sesiones gratuitas de hasta 30 h semanales), con precisión mixta (fp16) y los mismos scripts y particiones; las predicciones se descargaron para el ensamble.

- ***Tabla 2. Lenguajes, bibliotecas y paquetes utilizados.***

Herramienta	Versión	Uso
Python	3.14 (local) · 3.13 (Kaggle)	Lenguaje de todo el proyecto
pandas / NumPy / SciPy	3.0.3 / 2.5.1 / 1.18.0	Manipulación de datos y matrices dispersas
scikit-learn	1.9.0	TF-IDF, modelos lineales, validación y métricas (Pedregosa et al., 2011)
PyTorch	2.x	Entrenamiento de los Transformers
Hugging Face Transformers	4.x–5.x	Modelos preentrenados DeBERTa-v3 y MathBERT
Matplotlib / Seaborn	3.11.0 / 0.13.2	Figuras estáticas del informe
Streamlit / Plotly	1.65.0 / 7.1.0	Aplicación web y gráficas interactivas
joblib	1.5.3	Persistencia de modelos y paralelización
Kaggle CLI	2.2.4	Descarga de datos y ejecución en GPU
Git / GitHub	—	Control de versiones y contribuciones individuales

5. Resultados y análisis de resultados

5.1 Características del conjunto de datos original

El archivo `train.csv` contiene **36 696 observaciones y 7 variables**, con 15 preguntas, 49 opciones distintas, 35 concepciones erróneas y 65 etiquetas compuestas. No hay variables cuantitativas nativas: las variables numéricas del análisis se derivaron del texto.

Tabla 3. Diccionario de variables de train.csv.

Variable	Tipo	Valores únicos	Nulos	Rol
row_id	Identificador	36 696	0	Índice, sin valor predictivo
QuestionId	Categórica nominal	15	0	Contexto / agrupamiento
QuestionText	Texto	15	0	Enunciado de la pregunta
MC_Answer	Categórica (LaTeX)	49	0	Opción elegida
StudentExplanation	Texto libre	35 726	0	Predictora principal
Category	Categórica nominal	6	0	Objetivo (parte 1)
Misconception	Categórica nominal	35	26 836 (73.1 %)	Objetivo (parte 2)

`Category` tiene la forma `Veracidad_Tipo`: la veracidad (`True`/`False`) indica si la opción es correcta y el tipo (`Correct`, `Misconception`, `Neither`) describe la calidad del razonamiento. `Misconception` solo aplica cuando la categoría contiene una concepción errónea. El conjunto incluye además dos etiquetas casi homónimas, `Wrong_fraction` y `Wrong_Fraction`, que se conservaron tal como vienen porque corresponden a preguntas distintas.

5.2 Limpieza y preprocesamiento

Tabla 4. Decisiones de limpieza y preprocesamiento.

Situación detectada	Decisión	Justificación
26 836 nulos en Misconception (73.1 %)	Codificar como la categoría "NA"	Son estructurales: el valor existe si y solo si Tipo = Misconception; imputar inventaría etiquetas inexistentes.
736 filas iguales sin row_id; 970 textos repetidos	Conservar y agrupar en la partición	Pueden ser respuestas legítimas; agruparlas evita fuga entre entrenamiento y prueba.
LaTeX en el 96 % de MC_Answer y en los enunciados	Convertir a texto plano	Homogeneiza la notación numérica.
Mayúsculas, espacios y ortografía irregulares	Minúsculas y espacios unificados	Reduce variación superficial; los n-gramas de caracteres toleran errores.
78.3 % del texto con dígitos; operadores y negaciones	No eliminar	Pueden cambiar por completo el razonamiento matemático.
≈ 4 % de explicaciones atípicas por longitud	Auditar y conservar	Son válidas: explicaciones paso a paso o respuestas mínimas (señal de Neither).

Estas transformaciones están en `map_modelos.py`, que comparten el entrenamiento y la aplicación; así, una explicación nueva se preprocesa de forma idéntica e inadvertida para el usuario.

5.3 Análisis exploratorio

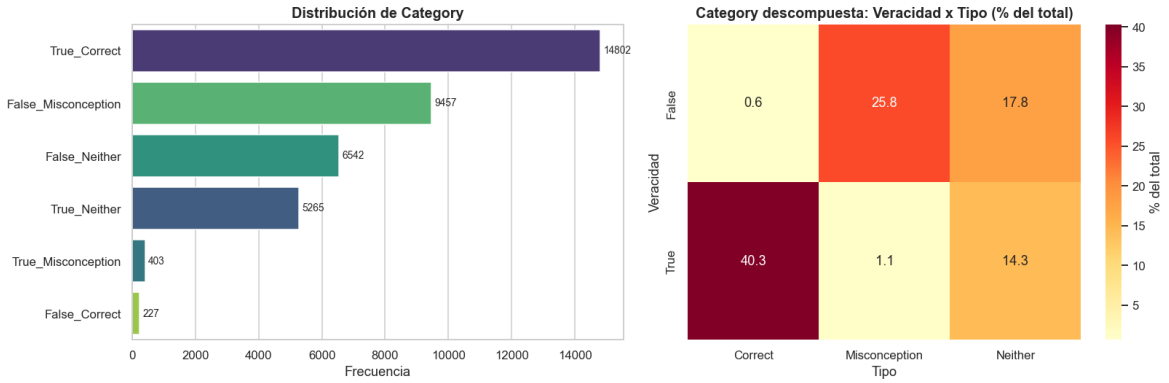


Figura 1. Distribución de Category y su descomposición en veracidad × tipo.

Una clase dominante (Figura 1). `True_Correct` concentra el 40.3 % de las observaciones, mientras que `True_Misconception` (1.1 %) y `False_Correct` (0.6 %) son muy raras, aunque son pedagógicamente las más interesantes.

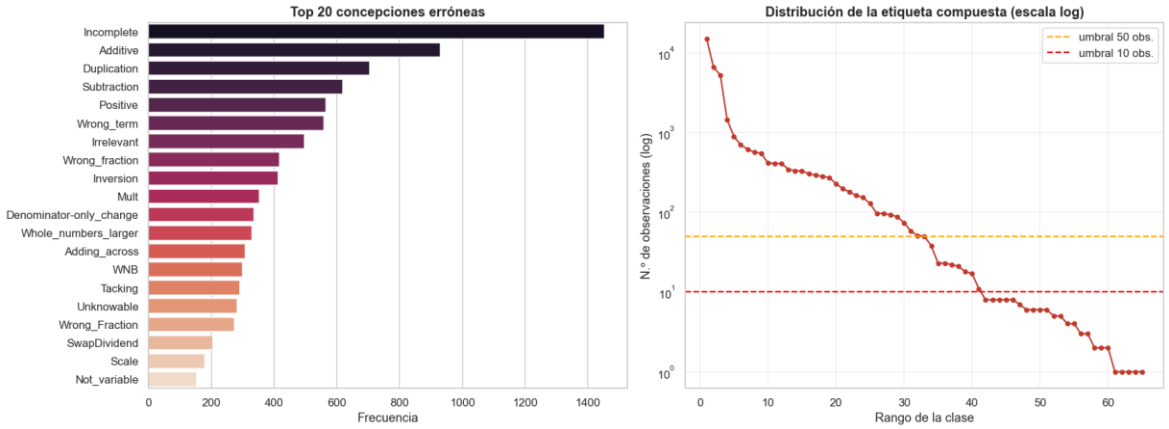


Figura 2. Concepciones erróneas más frecuentes y cola larga de la etiqueta compuesta.

Cola larga (Figura 2). 32 de las 65 etiquetas tienen menos de 50 casos y 24 menos de 10. Por eso se usan MAP@3 y macro-F1 además de la exactitud.

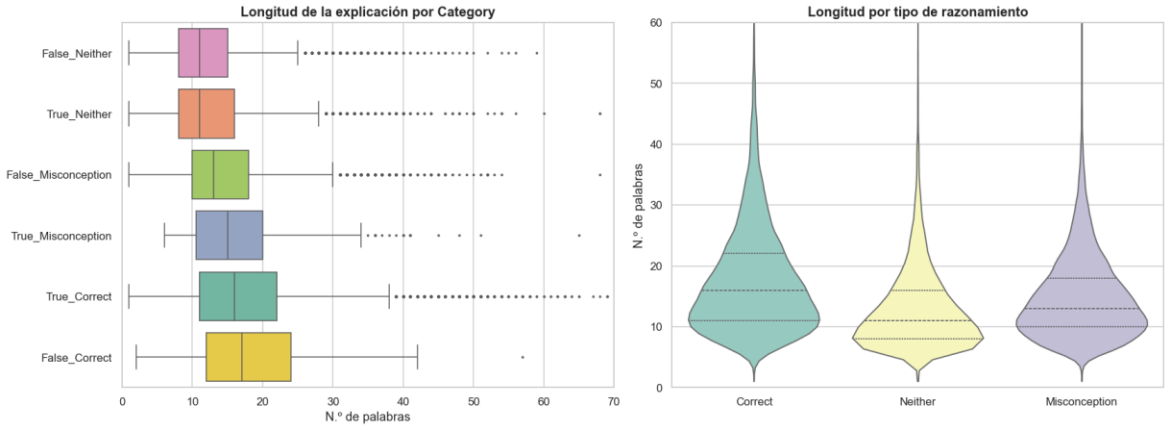


Figura 3. Longitud de la explicación por categoría y por tipo de razonamiento.

La longitud no explica por sí sola la etiqueta (Figura 3). Mediana de 13 palabras y máximo de 177. Los razonamientos correctos son más largos (mediana 16) que los erróneos (13) y los no evaluables (11), pero las distribuciones se solapan.

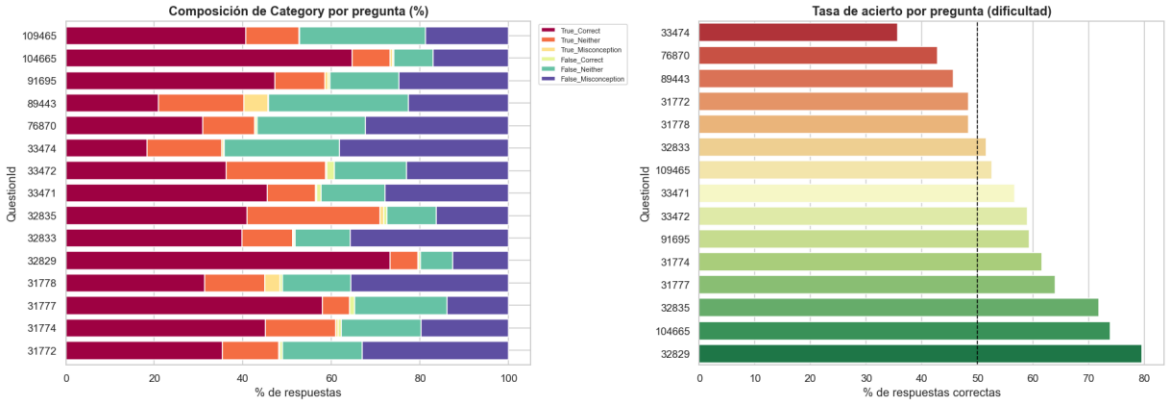


Figura 4. Composición de Category y tasa de acierto por pregunta.

La pregunta estructura el espacio de errores (Figura 4). La tasa de acierto varía entre 35.7 % y 79.6 %, 32 de las 35 concepciones aparecen en una sola pregunta y el par (pregunta, opción) determina la veracidad en 58 de 60 casos. Este hallazgo motivó el modelo jerárquico, la ablación del contexto y la validación por preguntas no vistas.

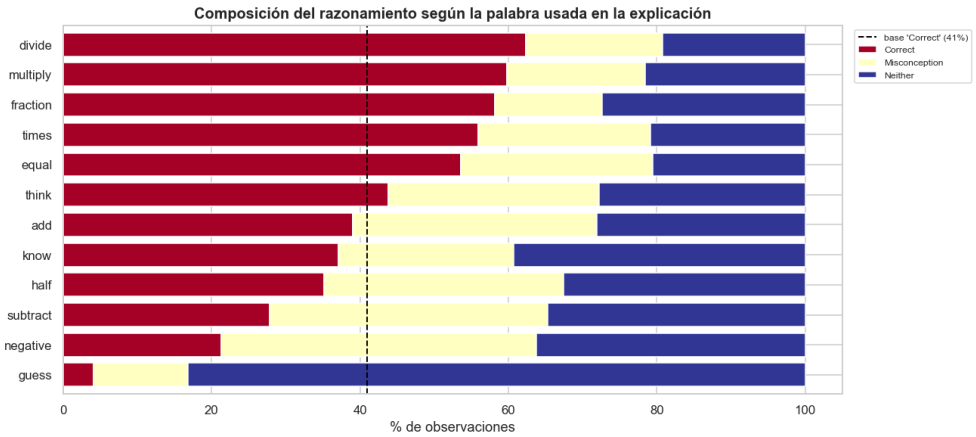


Figura 5. Tipo de razonamiento según palabras marcadoras.

El vocabulario discrimina (Figura 5). Verbos de procedimiento (*divide*, *multiply*) se asocian con razonamiento correcto y marcadores de incertidumbre (*guess*, *think*) con `Neither`, lo que justifica una línea base léxica.

5.4 Ajuste de hiperparámetros

Modelos lineales. Búsqueda en malla con validación cruzada de 3 pliegues (`StratifiedGroupKFold`) dentro del entrenamiento, con MAP@3 como criterio.

Tabla 5. Búsqueda de hiperparámetros de los modelos lineales (3 pliegues).

Modelo	Hiperparámetros	MAP@3 CV (media ± desv.)	Elegido
TF-IDF + SVM lineal	C = 0.1, class_weight = ninguno	0.9075 ± 0.0010	
TF-IDF + SVM lineal	C = 0.1, class_weight = balanced	0.8495 ± 0.0031	
TF-IDF + SVM lineal	C = 0.3, class_weight = ninguno	0.9107 ± 0.0009	✓
TF-IDF + SVM lineal	C = 0.3, class_weight = balanced	0.8791 ± 0.0022	
TF-IDF + SVM lineal	C = 1.0, class_weight = ninguno	0.9073 ± 0.0023	
TF-IDF + SVM lineal	C = 1.0, class_weight = balanced	0.8919 ± 0.0028	
TF-IDF + Regresión logística	C = 1.0, class_weight = ninguno	0.9130 ± 0.0007	
TF-IDF + Regresión logística	C = 4.0, class_weight = ninguno	0.9150 ± 0.0017	✓
TF-IDF + Regresión logística	C = 16.0, class_weight = ninguno	0.9132 ± 0.0019	
TF-IDF + Regresión logística	C = 4.0, class_weight = balanced	0.8700 ± 0.0031	

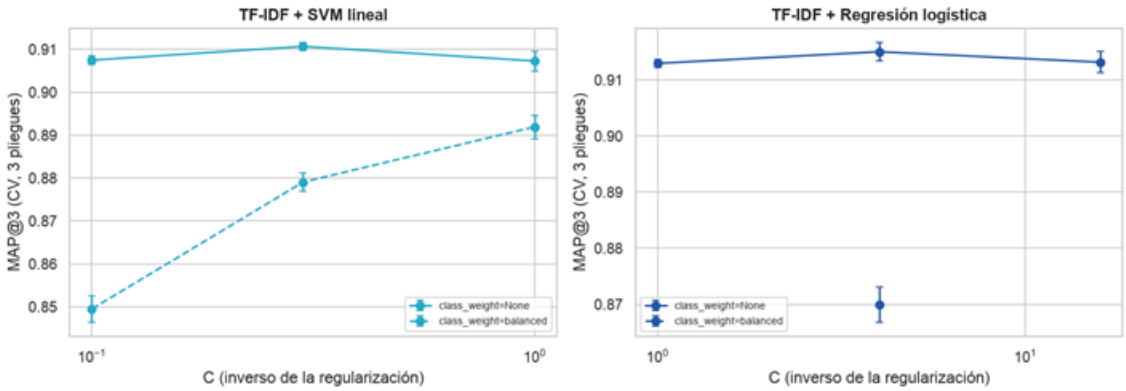


Figura 6. MAP@3 en validación cruzada según la regularización C y la ponderación de clases.

La Figura 6 muestra mesetas amplias: la regresión logística alcanza su máximo con **C = 4** y la SVM con **C = 0.3**, con diferencias menores a 0.004 entre valores vecinos. La **ponderación balanceada de clases empeoró el MAP@3** en ambos modelos (hasta 6 puntos): favorecer las clases raras degrada el orden de los candidatos frecuentes, que dominan la métrica. Se descartó.

Transformers. Por su costo, no se hizo búsqueda en malla. Se fijaron valores habituales para ajuste fino — tasa de aprendizaje con calentamiento lineal del 10 %, lotes de 16, decaimiento de pesos 0.01, recorte de gradiente en 1 y longitud máxima de 160 tokens— y se eligió la **época** con mejor MAP@3 en la partición de validación (parada temprana).

Tabla 6. Ajuste de los Transformers.

Modelo	Base preentrenada	Tasa de aprendizaje	MAP@3 val. por época
DeBERTa-v3	microsoft/deberta-v3-base	2e-05	0.872 → 0.899 → 0.912
MathBERT	tbs17/MathBERT	3e-05	0.899 → 0.914 → 0.927

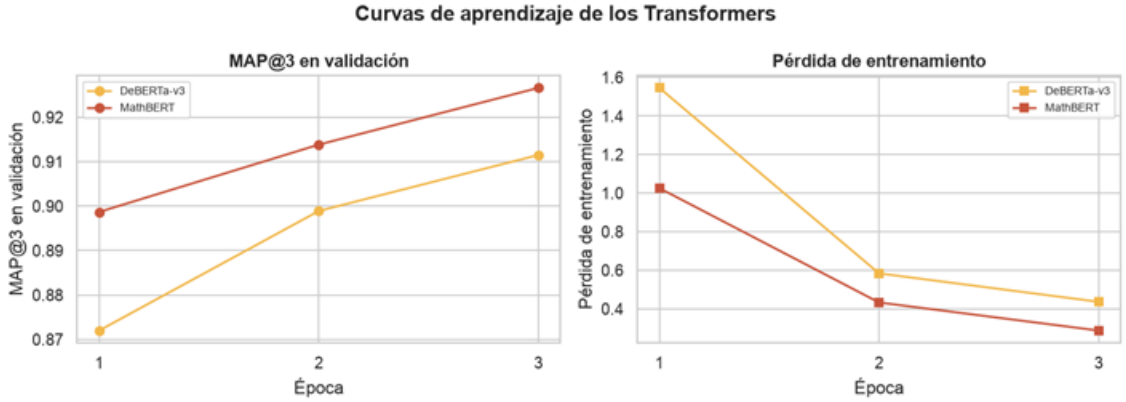


Figura 7. Curvas de aprendizaje de los Transformers.

5.5 Comparación de los algoritmos

Tabla 7. Desempeño y tiempos en el conjunto de prueba (7 339 explicaciones).

Modelo	MAP@3	Exactitud	Top-3	Macro-F1	F1 pond.	Entren. (s)	Pred. (s)
Ensamble calibrado	0.930	0.868	0.995	0.531	0.864	512	5.53
MathBERT	0.926	0.862	0.993	0.500	0.857	455	4.96
TF-IDF + Regresión logística jerárquica	0.921	0.851	0.995	0.522	0.845	57	0.58
TF-IDF + SVM lineal	0.919	0.855	0.986	0.522	0.849	10	0.44
DeBERTa-v3	0.914	0.847	0.986	0.407	0.838	822	9.44
Prior (pregunta+opción)	0.838	0.695	0.989	0.280	0.613	0	0.01

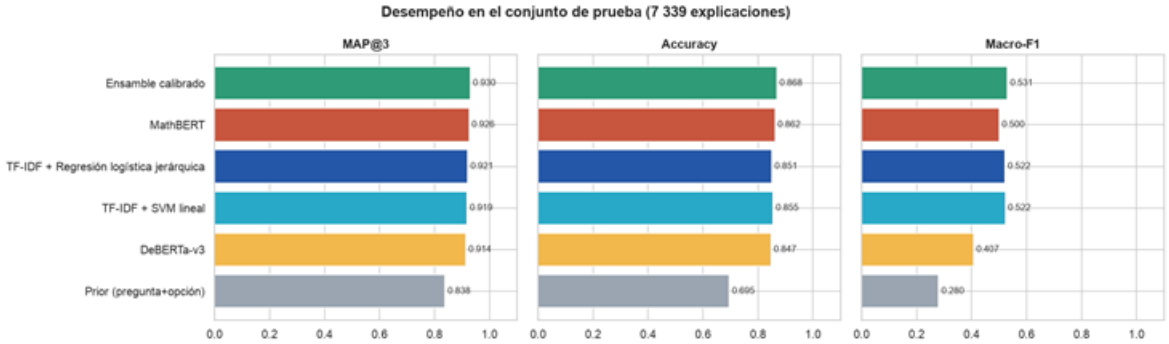


Figura 8. MAP@3, exactitud y macro-F1 en el conjunto de prueba.

Líneas base (Figura 8). Leer la explicación aporta: la regresión logística jerárquica obtiene MAP@3 = **0.921** frente a 0.838 del prior, con 16 puntos más de exactitud y casi el doble de macro-F1 (0.280 → 0.522). La SVM queda prácticamente empatada (MAP@3 = 0.919) con un entrenamiento unas 6 veces más rápido; la regresión logística ordena mejor el segundo y el tercer candidato (exactitud top-3 0.995 frente a 0.986).

Transformers. DeBERTa-v3 alcanza MAP@3 = 0.914, exactitud = 0.847 y macro-F1 = 0.407, con 14 min de entrenamiento en GPU; **MathBERT** alcanza MAP@3 = 0.926, exactitud = 0.862 y macro-F1 = 0.500, con 8 min de entrenamiento en GPU.

Ensamble calibrado. Con $\alpha = 0.65$ (peso de MathBERT) elegido en validación y la restricción jerárquica, el ensamble obtiene MAP@3 = **0.930**, exactitud = 0.868 y macro-F1 = 0.531.

Discusión. MathBERT es el mejor modelo individual (MAP@3 0.926) y supera a la regresión logística (0.921). DeBERTa-v3-base, en cambio, queda por debajo de las líneas base (MAP@3 0.914) y tiene el macro-F1 más bajo de los modelos de texto (0.407): su MAP@3 de validación seguía creciendo en la tercera época y su pérdida de entrenamiento era aún alta, señal de que tres épocas no bastan para un modelo de ese tamaño con 65 clases desbalanceadas. MathBERT, más pequeño y preentrenado con textos de educación matemática, converge antes y aprende mejor las clases raras, por lo que se adoptó como el Transformer del ensamble: la adaptación al dominio mejoró tanto el macro-F1 como la generalización respecto de DeBERTa (sección 5.7). El **ensamble calibrado** combina la evidencia contextual de MathBERT (65.0 % del peso) con la léxica de la regresión logística y obtiene el mejor resultado en todas las métricas

principales: MAP@3 **0.930**, exactitud 0.868 y macro-F1 0.531. La mejora sobre cada componente por separado indica que ambas representaciones cometen errores distintos.

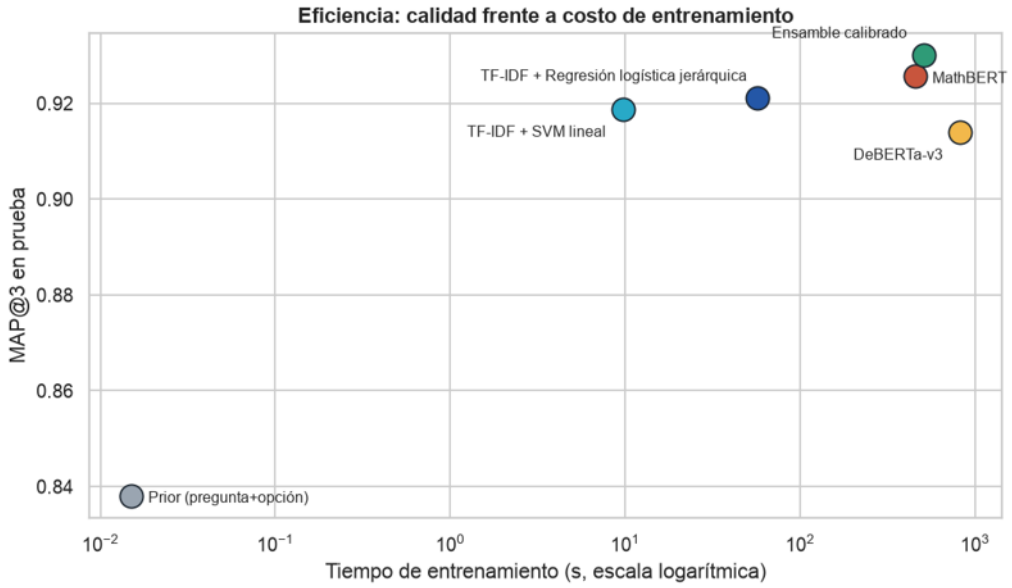


Figura 9. Calidad (MAP@3) frente a tiempo de entrenamiento.

Tiempos (Figura 9). Los modelos lineales entrenan en segundos o minutos en CPU y predicen más de 7 000 explicaciones en menos de 1 s. Los Transformers requieren entre 8 y 14 minutos de GPU para entrenar y unos 5–10 s para predecir el conjunto de prueba (alrededor de 1 ms por explicación). MathBERT ofrece la mejor relación calidad-costo entre los Transformers: es más rápido que DeBERTa y más preciso. En la aplicación, todos responden en tiempo interactivo para una explicación.

5.6 Ablación del contexto de entrada

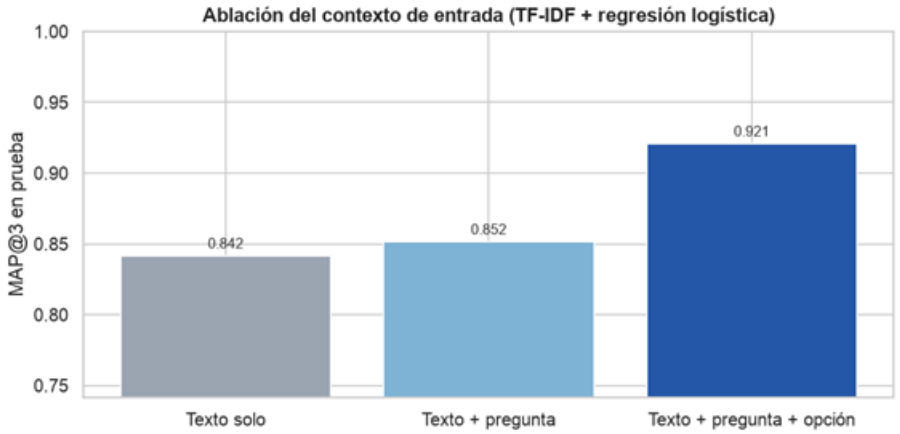


Figura 10. MAP@3 de la regresión logística según el contexto de entrada.

Con preguntas conocidas, el contexto suma (Figura 10): texto solo 0.842, + pregunta 0.852 y + pregunta y opción **0.921**. La opción elegida es la fuente más valiosa porque fija la veracidad, que el texto por sí solo no siempre permite inferir.

5.7 Generalización a preguntas no vistas (validación B)

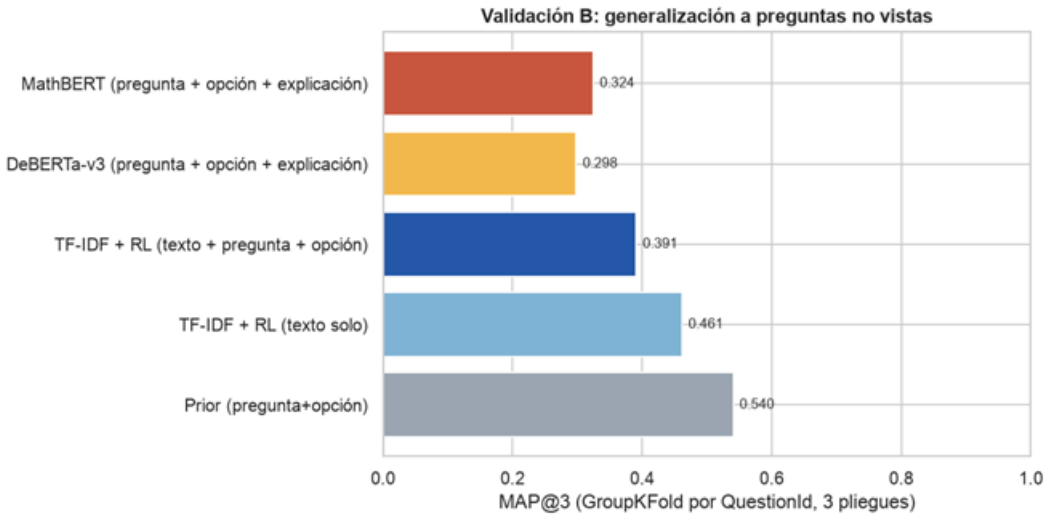


Figura 11. MAP@3 con preguntas no vistas (GroupKFold por QuestionId).

Con preguntas completas fuera del entrenamiento (Figura 11) todos los modelos caen con fuerza. El prior se reduce a proponer las etiquetas globales más frecuentes (0.540). La regresión logística con texto solo obtiene 0.461, **mejor que la misma regresión con pregunta y opción** (0.391).

Este resultado confirma el riesgo de usar `QuestionId` como atajo: el identificador mejora el desempeño con preguntas conocidas, pero empeora la transferencia, porque el modelo aprende asociaciones pregunta → concepción que no existen en una pregunta nueva. Coincide con Padó et al. (2024). Como el

91 % de las concepciones de una pregunta nueva no aparecen en entrenamiento, ningún modelo puede acertarlas: la validación B mide sobre todo la capacidad de reconocer la categoría general.

Los Transformers tampoco resuelven la transferencia: con preguntas no vistas obtienen MAP@3 de 0.324 (MathBERT) y 0.298 (DeBERTa), por debajo de la regresión logística con texto solo. Aunque reciben el texto de la pregunta y no su identificador, aprenden igualmente asociaciones entre el enunciado y las concepciones de esa pregunta. El mejor valor lo obtiene el prior, que en una pregunta nueva se limita a proponer las etiquetas globales más frecuentes (`True_Correct:NA`, `False_Neither:NA`, etc.): con solo 15 preguntas, ningún modelo aprendió patrones de razonamiento que transfieran mejor que esa heurística.

5.8 Calibración y abstención

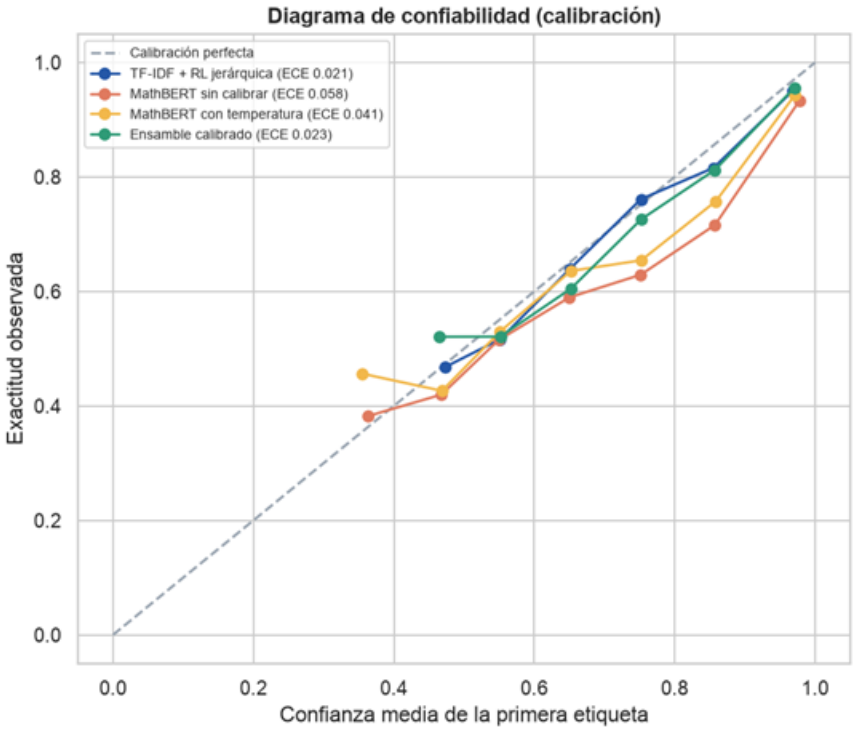


Figura 12. Diagrama de confiabilidad de los modelos probabilísticos.

La Figura 12 compara la confianza declarada con la exactitud observada (ECE: TF-IDF + Regresión logística jerárquica: 0.021; DeBERTa-v3 (sin calibrar): 0.050; DeBERTa-v3 (temperatura): 0.037; MathBERT (sin calibrar): 0.058; MathBERT (temperatura): 0.041; Ensamble calibrado: 0.023). El escalado de temperatura (T = 1.10) corrige la sobreconfianza típica de las redes neuronales (Guo et al., 2017).

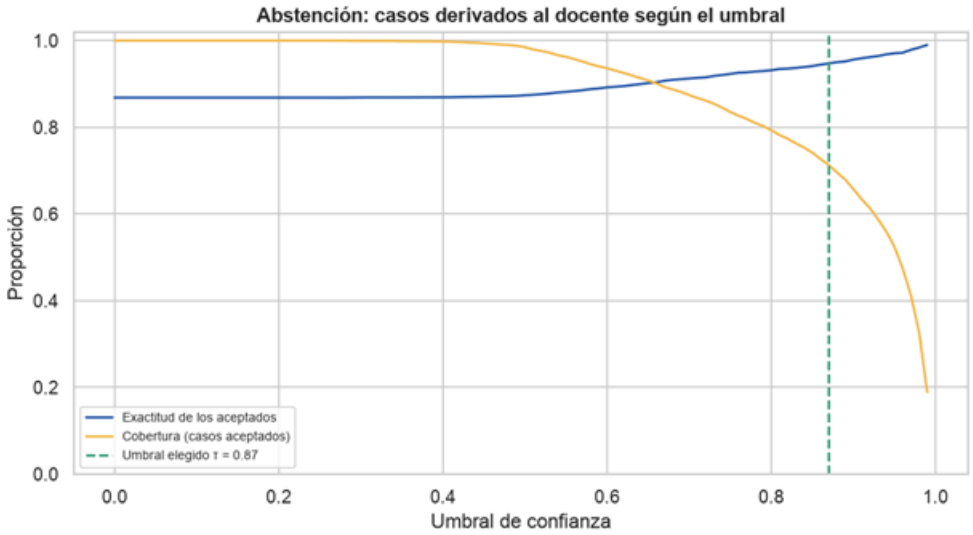


Figura 13. Exactitud de los casos aceptados y cobertura según el umbral de confianza.

Abstención (Figura 13). Se eligió en validación el menor umbral cuya exactitud sobre los casos aceptados es de al menos 95 %: **τ = 0.87**. En prueba, el sistema decide automáticamente el 71.3 % de los casos con una exactitud de 94.7 %, y deriva al docente el resto, donde la exactitud sería solo de 67.3 %. Así, la revisión humana se concentra donde el modelo es menos confiable.

5.9 Análisis de errores y selección del modelo

Tabla 8. Exactitud por nivel de la jerarquía (Ensamble calibrado).

Dimensión evaluada	Exactitud
Veracidad (True/False)	99.9 %
Category (6 clases)	87.2 %
Etiqueta compuesta (65 clases)	86.8 %
Misconception (solo filas con concepción)	87.5 %

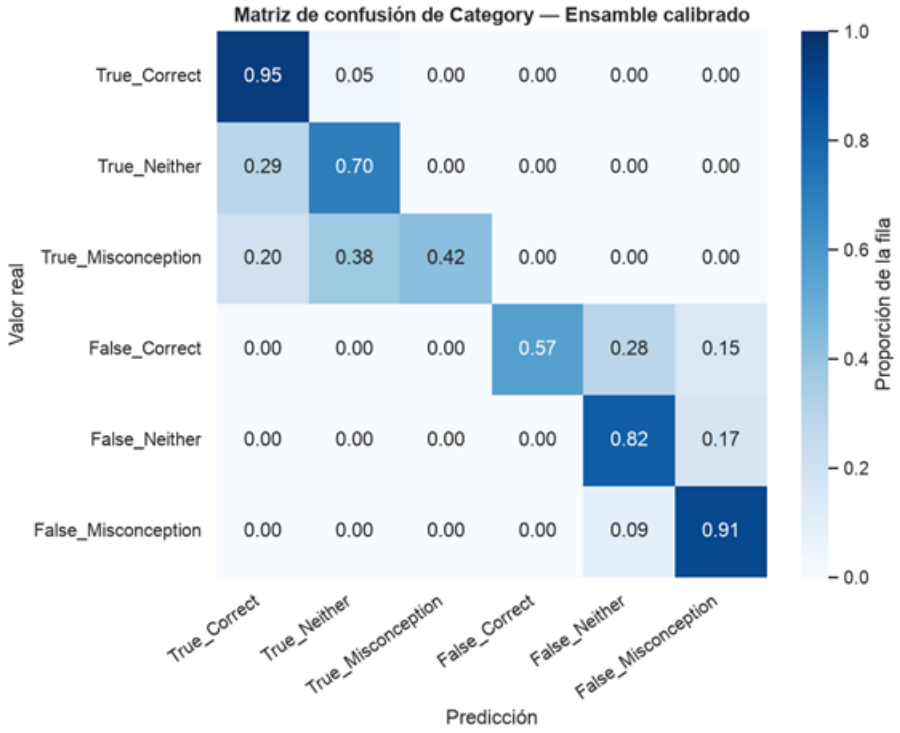


Figura 14. Matriz de confusión de Category (Ensamble calibrado).

La veracidad se predice casi perfectamente (99.9 %); la dificultad está en el tipo de razonamiento. La matriz (Figura 14) muestra dos bloques sin confusiones entre sí (`True_*` y `False_*`). Los errores se concentran en `True_Neither`, confundido con `True_Correct` (distinguir una explicación correcta pero vaga es subjetivo), en `True_Misconception` —acertar con un razonamiento erróneo, solo 403 casos— y en `False_Correct`.

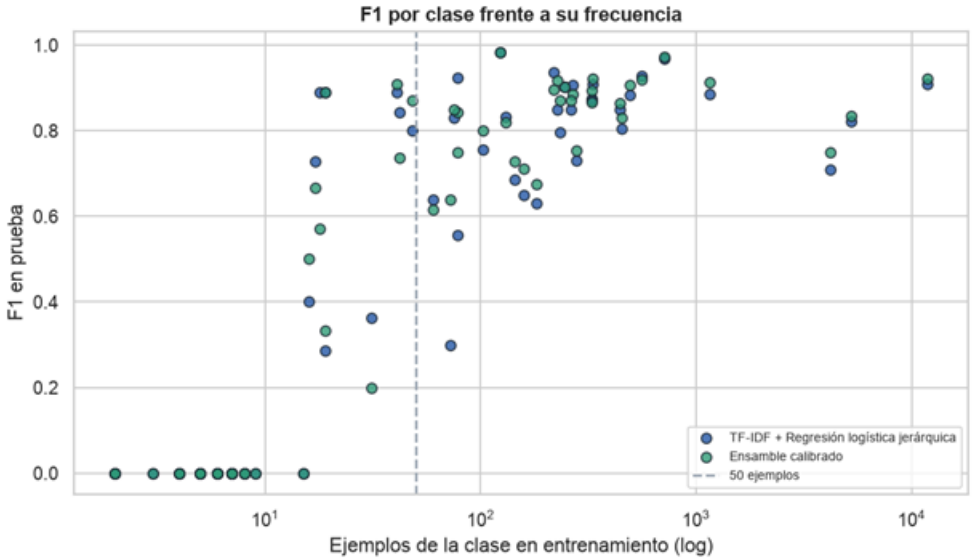


Figura 15. F1 por etiqueta frente a su número de ejemplos de entrenamiento.

La Figura 15 explica el macro-F1 moderado: las etiquetas con menos de unos 15 ejemplos de entrenamiento obtienen F1 cercano a cero, mientras que la mayoría de las que superan 100 ejemplos rebasan 0.8. El cuello de botella es la cola larga, más que la capacidad del algoritmo.

Selección del modelo. Se selecciona el **ensamble calibrado** (MathBERT con escalado de temperatura T = 1.10, α = 0.65, más la regresión logística jerárquica): mejor MAP@3 (0.930), mejor macro-F1 (0.531), buena calibración (ECE 0.023) y predicciones coherentes con la pregunta. Su costo es mayor —requiere GPU para entrenar y un modelo de unos 200 MB—, por lo que la **regresión logística jerárquica** se mantiene como alternativa ligera: solo 0.009 menos de MAP@3, entrenamiento en un minuto en CPU y es la que funciona en la versión web de la aplicación. La SVM lineal es la opción más rápida de reentrenar. DeBERTa-v3 no se seleccionó: con el presupuesto de cómputo disponible no superó a las líneas base.

5.10 Aplicación

Se construyó una aplicación web con **Streamlit** (`app/app.py`) que permite clasificar explicaciones nuevas con los modelos entrenados, explorar el conjunto de datos limpio y comparar el rendimiento de los algoritmos. Se ejecuta localmente con `streamlit run app/app.py` y se publica en Streamlit Community Cloud directamente desde el repositorio de GitHub.

Clasificar una explicación. El usuario elige la pregunta y la opción marcada (la correcta aparece señalada), escribe la explicación o carga un ejemplo real de una categoría, y elige uno o varios modelos. Para cada modelo se muestran las tres etiquetas más probables con su probabilidad, una descripción en lenguaje natural de la categoría y, cuando la confianza es menor que el umbral τ, la recomendación de **derivar el caso al docente**. Una gráfica compara las respuestas de todos los modelos e indica si coinciden. El preprocesamiento (conversión de LaTeX, normalización y vectorización) ocurre de forma invisible.

Enlace a la aplicación:

https://proyecto-2grupo-1dssec-10-yakgdjmx6l95w8ozmt5yep.streamlit.app/

Deploy

MAP

Charting Student Math Misunderstandings · Grupo 1 · CC3084

Secciones

- Clasificar una explicación
- Explorar los datos
- Rendimiento de los modelos
- Acerca del proyecto

Las predicciones orientan la revisión docente; no son un diagnóstico definitivo.

Clasificar la explicación de un estudiante

Elija la pregunta y la opción que marcó el estudiante, escriba su explicación **en inglés** (idioma del conjunto de entrenamiento) y el sistema propondrá las tres etiquetas más probables.

Pregunta

31772 · What fraction of the shape is not shaded? Give your answer in its simplest form. [Imz

Opción elegida

1/3 ✓ 3/6 3/8 3/9

Ejemplos reales

Cargar un ejemplo de la categoría

False_Correct

Cargar ejemplo

Explicación del estudiante

threee of them are not shaded out of the nine.

Modelos a utilizar

Prior (pregunta+opción) × TF-IDF + SVM lineal × TF-IDF + Regresión logística jerárquica × DeBERTa-v3 × MathBERT × Ensamble calibrado ×

Clasificar

Etiqueta real de este ejemplo: **False_Correct:NA**

Prior (pregunta+opción)

1. False_Misconception · concepción errónea: **Incomplete** (66.5%)
Opción incorrecta debida a una concepción errónea.

2. False_Neither (30.7%)
Opción incorrecta y explicación no evaluable.

3. False_Misconception · concepción errónea: **WNB** (1.6%)
Opción incorrecta debida a una concepción errónea.

TF-IDF + SVM lineal

1. False_Misconception · concepción errónea: **Incomplete** (0.81)
Opción incorrecta debida a una concepción errónea.

2. False_Correct (0.09)
Opción incorrecta, aunque el razonamiento es correcto (descuido).

3. False_Neither (0.01)
Opción incorrecta y explicación no evaluable.

Puntaje relativo de la SVM (no es una probabilidad calibrada).

TF-IDF + Regresión logística jerárquica

1. False_Misconception · concepción errónea: **Incomplete** (45.0%)
Opción incorrecta debida a una concepción errónea.

2. False_Correct (36.9%)
Opción incorrecta, aunque el razonamiento es correcto (descuido).

3. False_Neither (14.3%)
Opción incorrecta y explicación no evaluable.

Confianza 45% < 87%: se recomienda **derivar al docente**.

DeBERTa-v3

1. False_Misconception · concepción errónea: **Incomplete** (97.3%)
Opción incorrecta debida a una concepción errónea.

2. False_Neither (1.5%)
Opción incorrecta y explicación no evaluable.

3. False_Misconception · concepción errónea: **WNB** (0.2%)
Opción incorrecta debida a una concepción errónea.

Confianza 87% ≥ umbral 87%.

MathBERT

1. False_Misconception · concepción errónea: **Incomplete** (96.5%)
Opción incorrecta debida a una concepción errónea.

2. False_Neither (2.2%)
Opción incorrecta y explicación no evaluable.

3. False_Correct (0.3%)
Opción incorrecta, aunque el razonamiento es correcto (descuido).

Confianza 96% ≥ umbral 87%.

Ensamble calibrado

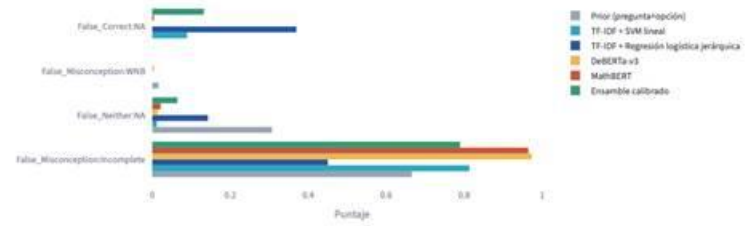
1. False_Misconception · concepción errónea: **Incomplete** (79.0%)
Opción incorrecta debida a una concepción errónea.

2. False_Correct (13.2%)
Opción incorrecta, aunque el razonamiento es correcto (descuido).

3. False_Neither (6.4%)
Opción incorrecta y explicación no evaluable.

Confianza 79% < 87%: se recomienda **derivar al docente**.

Comparación entre modelos



Todos los modelos coinciden en la primera etiqueta: **False_Misconception:Incomplete**.

Figura 16. Aplicación: clasificación de una explicación con varios modelos.

Explorar los datos. Filtros por pregunta y categoría, indicadores clave y gráficas interactivas de la distribución de etiquetas, la cola larga, las variables derivadas del texto (longitud, dígitos, símbolos, diversidad léxica) y la composición por pregunta, acompañadas de los hallazgos que explican el comportamiento de los modelos.

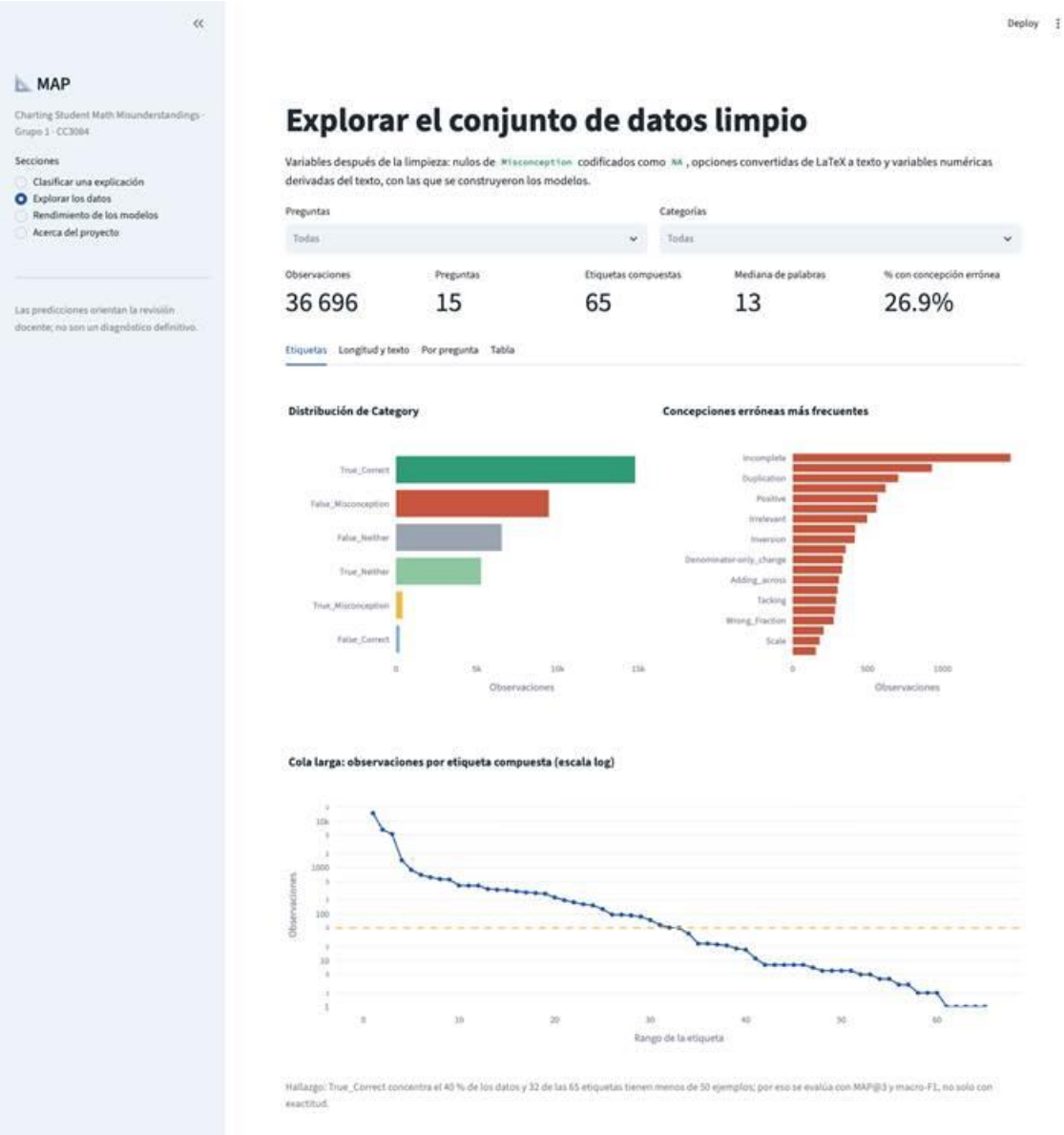


Figura 17. Aplicación: exploración interactiva del conjunto limpio.

Rendimiento de los modelos. Gráficas interactivas de cada métrica en el conjunto de prueba, calidad frente a tiempo, validación por preguntas no vistas, matriz de confusión, ablación del contexto, curva de abstención, calibración y búsqueda de hiperparámetros. Un interruptor permite **ocultar o mostrar** esta información y un selector elige qué modelos comparar.

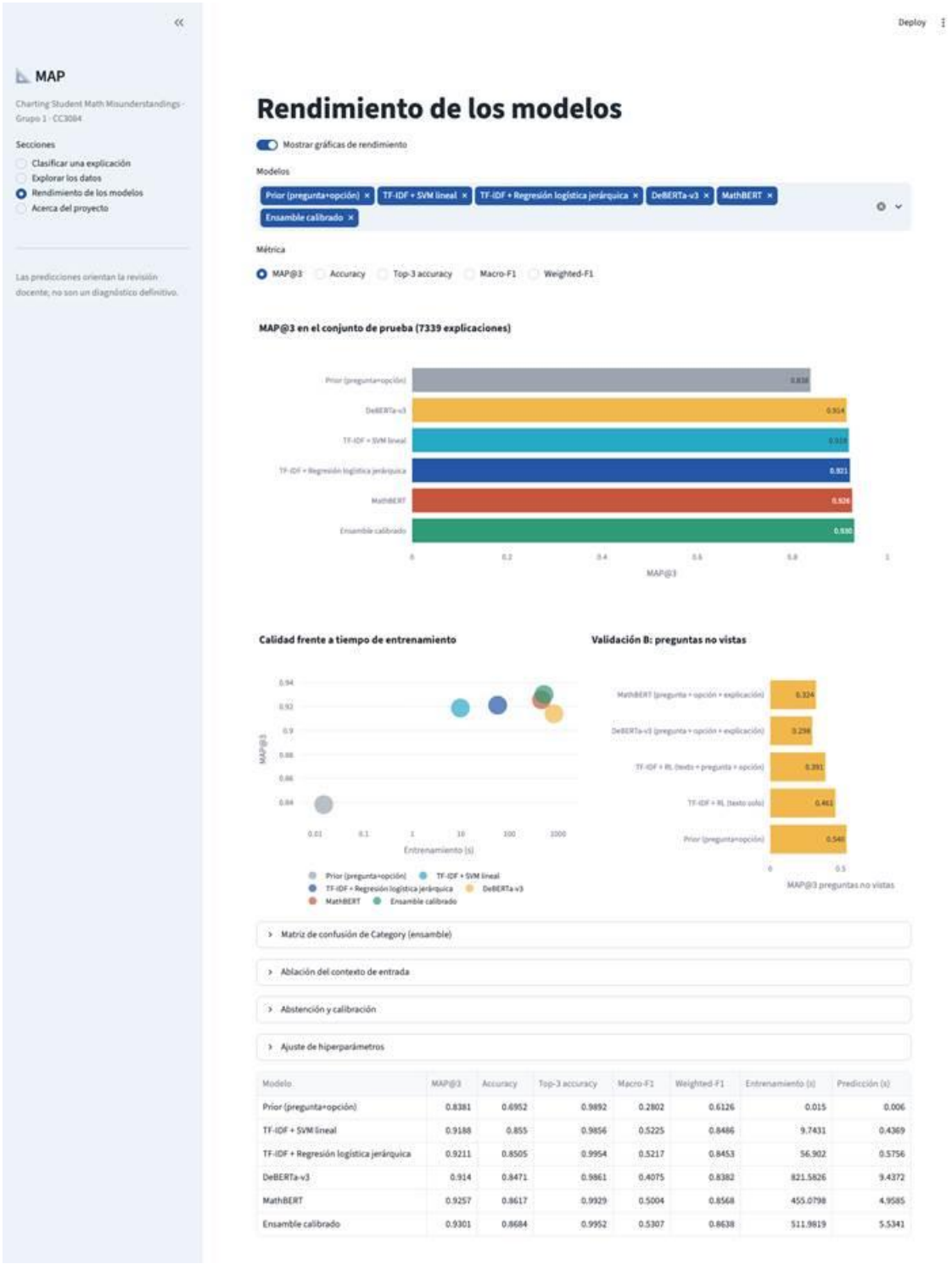


Figura 18. Aplicación: comparación interactiva del rendimiento.

Selección de tecnologías. Streamlit permite construir la interfaz en Python, el mismo lenguaje de los modelos, y cargar directamente los objetos de scikit-learn y PyTorch sin una capa de servicio adicional; además se despliega gratis desde GitHub. Plotly aporta gráficas interactivas (zoom, filtrado y valores al pasar el cursor). La paleta usa el **azul institucional** como color principal, acentos análogos (cian y verde)

para los modelos y un **complementario cálido** (ámbar) para resaltar; los grises se reservan para la referencia. Los colores de las categorías siguen una convención semántica (verde para razonamiento correcto, rojo para concepción errónea) y el tema claro garantiza contraste suficiente del texto.

En la versión publicada no se distribuyen los textos originales de la competencia, conforme a sus reglas: la exploración usa las variables derivadas y el usuario escribe sus propias explicaciones. Los ejemplos reales solo están disponibles al ejecutar la aplicación con los datos descargados de Kaggle. Además, los Transformers ajustados (entre 200 y 360 MB) superan el límite de archivos de GitHub, por lo que la versión web ofrece los modelos TF-IDF y la ejecución local, los seis modelos.

6. Conclusiones

- **Revisión y selección (objetivos 1 y 2).** La literatura de calificación automática orientó un esquema incremental: líneas base léxicas, un Transformer contextual, adaptación al dominio y un ensamble calibrado, adoptando complejidad solo si mejora el MAP@3 y el macro-F1 bajo el mismo protocolo. El análisis exploratorio mostró que el problema es contextual, jerárquico y muy desbalanceado, y que los nulos de `Misconception` son estructurales.
- **Modelos y evaluación (objetivos 3 y 4).** Las líneas base TF-IDF alcanzan MAP@3 = 0.921 (regresión logística jerárquica) y 0.919 (SVM), muy por encima del prior sin texto (0.838). MathBERT, adaptado al dominio, fue el mejor modelo individual (0.926); DeBERTa-v3 (0.914) no superó a las líneas base con tres épocas. El **ensamble calibrado** de MathBERT y la regresión logística alcanzó el mejor desempeño: MAP@3 **0.930**, exactitud 86.8 % sobre 65 clases y macro-F1 0.531. La complejidad adicional se justificó solo donde mejoró ambas métricas.
- **Contexto y generalización.** La pregunta y la opción elevan el MAP@3 de 0.842 a 0.921 con preguntas conocidas, pero con preguntas no vistas el desempeño cae a 0.391 y el modelo de texto solo generaliza mejor: el identificador de la pregunta funciona como atajo. El sistema es adecuado para el banco de preguntas conocido, no para preguntas nuevas sin datos etiquetados.
- **Confiabilidad (objetivo 5).** Con un umbral $\tau = 0.87$, el sistema decide solo el 71.3 % de los casos con 94.7 % de exactitud y deriva el resto al docente: un clasificador que prioriza casos y no reemplaza el juicio docente.
- **Aplicación (objetivo 6).** La aplicación Streamlit clasifica explicaciones nuevas con uno o todos los modelos, explica las categorías, recomienda la abstención y permite explorar los datos y el rendimiento con gráficas interactivas que pueden ocultarse.

Efectividad de los algoritmos. Para ordenar las tres etiquetas más probables en preguntas conocidas, los modelos son eficaces; sus debilidades son las etiquetas con muy pocos ejemplos, las categorías pedagógicamente sutiles (`True_Misconception`, `True_Neither`) y las preguntas nuevas. Por ello las predicciones deben usarse para **priorizar la revisión docente**, no como diagnóstico definitivo.

Trabajo futuro

- Incorporar descripciones textuales de cada concepción errónea para comparar la explicación con ellas y mejorar la transferencia a preguntas nuevas.
- Aumento conservador de datos para las clases raras, revisado por docentes.
- Pruebas de robustez: perturbaciones inocuas (palabras irrelevantes, ortografía) y cambios matemáticamente decisivos (números, signos, negaciones) (Filighera et al., 2024).
- Ampliar el banco de preguntas para medir con más precisión la generalización.

Referencias

Kaggle. (2025). *MAP—Charting student math misunderstandings* [Competencia de ciencia de datos]. https://www.kaggle.com/competitions/map-charting-student-math-misunderstandings

Morley, F., Walland, E., & Vidal Rodeiro, C. L. (2026). Auto-marking short answer questions in science: The foundational years of transformer-based models from BERT to GPT-4. *International Journal of Artificial Intelligence in Education, 36*(1–2), 100005. https://doi.org/10.1016/j.ijaied.2026.100005

Padó, U., Eryilmaz, Y., & Kirschner, L. (2024). Short-answer grading for German: Addressing the challenges. *International Journal of Artificial Intelligence in Education, 34*(4), 1321–1352. https://doi.org/10.1007/s40593-023-00383-w

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Rennie, J. D. M., Shih, L., Teevan, J., & Karger, D. R. (2003). Tackling the poor assumptions of naive Bayes text classifiers. En *Proceedings of the Twentieth International Conference on Machine Learning* (pp. 616–623). AAAI Press.

Rittle-Johnson, B., Adler, R., Durkin, K., Burleigh, L., King, J., & Crossley, S. (2025). Detecting math misconceptions: An AI benchmark dataset. En *Proceedings of the 1st Workshop on Advancing Artificial Intelligence through Theory of Mind* (pp. 20–24). Association for Computational Linguistics. https://aclanthology.org/2025.aimecon-wip.3/

Rushton, S. J. (2018). Teaching and learning mathematics through error analysis. *Fields Mathematics Education Journal, 3*, Artículo 4. https://doi.org/10.1186/s40928-018-0009-y

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0

Enlace al Repositorio de GitHub

https://github.com/DanielBarillasM/Proyecto-2_Grupo-1_DS_Sec-10.git

Enlace al documento del informe en OneDrive

informe_resultados_MAP.docx